Índice

Protocolo-base sugerido — caso feliz e casos de erro (Saga coreografada × orquestrada) 1
DOCUMENTO DE TRABALHO — PROPOSTA INICIAL — NÃO OFICIAL . . 1
Como ler os rótulos . . . 2
1. Objetivo, escopo e regras de equivalência . . . 2
2. Cenário de liquidação de ordem . . . 4
3. Caso feliz: passo a passo, estados, identificadores e invariantes . . . 7
4. Protocolo sugerido de coordenação . . . 10
5. Compensações em ordem inversa e resultado esperado . . . 11
6. Matriz de casos de erro/falha (proposta) . . . 13
7. Retries, idempotência, DLQ/outbox (decisões a validar) . . . 15
8. Métricas operacionais — como medir (sem resultados) . . . 16
9. Papel das ferramentas (confirmadas e candidatas) . . . 19
10. Matriz inicial de execução e checklist de validação . . . 21
11. Itens ainda a confirmar e referências . . . 22

Protocolo-base sugerido — caso feliz e casos de erro (Saga coreografada × orquestrada)

DOCUMENTO DE TRABALHO — PROPOSTA INICIAL — NÃO OFICIAL

- **Status:** `DOCUMENTO DE TRABALHO - PROPOSTA INICIAL - NÃO OFICIAL`.
- **Nenhum experimento foi executado** no projeto até o momento e **nenhum resultado experimental** é afirmado aqui. Este texto é uma **sugestão inicial de protocolo-base** para o **caso feliz** e os **casos de erro** do experimento Saga; não é o protocolo aprovado.
- **Depende de validação do(a) orientador(a).** As decisões de desenho já **confirmadas** estão marcadas **[C]** (§§3.3–3.4 e §3.5); os **nomes, contratos, limiares, janelas e parâmetros** ainda em aberto permanecem **`[A CONFIRMAR]`** até essa validação (§3.10 do plano de trabalho).
- **Não substitui** o plano de trabalho nem o protocolo experimental oficial de outubro/2026; **complementa** o plano com um recorte operacional do **caminho feliz e das falhas**.
- **Derivado de** (sem nova busca bibliográfica): `docs/plano-de-trabalho-pesquisa-2026-2027.md` (§§3.1–3.10 e apêndices), `research/fichamentos/fichamento-fundamentos-saga.md`, `research/fichamentos/fichamento-2pc-saga.md`,

`research/fichamentos/fichamento-estudos-comparativos-saga.md` e `research/relatorio-validacao-escopo-2026-09-22.md`.
- **Registro interno:** `docs/confirmacoes.md` foi usado **apenas** como registro interno das confirmações já dadas pelo(a) orientador(a); **não** é citado aqui como documento para compartilhamento.
- **Não inventa** versões, resultados, limiares, contratos ou nomes como decisões. Nomes ilustrativos estão marcados `[A CONFIRMAR]`. Citações em **ABNT**.
- Este documento **não altera** o plano nem qualquer outro arquivo existente; o PDF correspondente, se gerado, é apenas renderização deste `.md`.

Como ler os rótulos

Padrão já usado nos documentos do repositório (`AGENTS.md`, seções 5 e 8):

- **[E] — Evidência verificada:** afirmação sustentada por fonte já registrada no repositório ou por decisão já documentada, com origem identificável.
- **[R] — Recomendação/proposta:** sugestão deste documento, ainda sujeita a decisão do(a) orientador(a).
- **[C] — Confirmado:** item de **desenho** dado como decidido para o protocolo-base nesta revisão: estados dos serviços (§3.3), invariantes de consistência e de concorrência (§3.4), premissa de ausência de isolamento inter-Saga (§3.5). **Não** é resultado experimental, evidência comparativa nem dispensa a validação final do(a) orientador(a).
- **[A CONFIRMAR] — Pendência:** dado, nome, contrato, versão, limiar, janela ou decisão que **não** pode ser tratado como confirmado. Nada assim marcado é fato, resultado ou decisão.

  Regra de ouro (plano §3, §3.10): nenhuma coleta de dados experimentais começa **antes** de o protocolo estar escrito, revisado e aprovado. Este documento é um **insumo** para essa redação.

---

1. Objetivo, escopo e regras de equivalência

1.1 Objetivo

[R] Propor um **protocolo-base** para o **caminho feliz** e para os **casos de erro/falha** do experimento, de modo que as **duas variantes** do padrão Saga — **coreografada** e **orquestrada** — sejam exercitadas sobre **o mesmo fluxo de negócio**, com **os mesmos eventos/comandos**

conceituais e **as mesmas regras de equivalência**, isolando a **coordenação** como única diferença.

[**R**] O protocolo-base deve permitir, no futuro: (i) redigir os contratos (comandos/respostas e eventos/fatos) das duas variantes; (ii) definir o estado final esperado em sucesso, rejeição e compensação; (iii) preparar a matriz de injeção de falhas (§6) e as métricas operacionais (§8).

1.2 Escopo

Dimensão	Delimitação
Objeto	Um **único** fluxo de **liquidação de ordem** (plano §3.1).
Variantes	**Saga coreografada** × **Saga orquestrada** — nenhuma outra é braço experimental. [**E**] O 2PC é apenas **contraste conceitual**, não tratamento (`research/fichamentos/fichamento-2pc-saga.md`).
MVP	RQ2 (falhas/compensação/consistência) principal; RQ1 (desempenho) complementar (plano §2.1).
Não é	Documento oficial, protocolo aprovado, resultado ou fonte de números.
Generalização	Válido **para o cenário estudado**; sem extrapolação (plano §1.2).

1.3 Regras de equivalência entre as variantes

[**R**] As duas implementações devem diferir **apenas** no estilo de coordenação. Igualar (plano §3.3 e §3.8; relatório de validação §6.3):

Controle	Como igualar
Linguagem/framework	mesmas versões fixadas de **Java/Spring Boot** (`[A CONFIRMAR]` a versão exata).
Broker	**mesmo** broker (**RabbitMQ**) e mesma configuração comum.
Banco	**PostgreSQL** como mesma tecnologia; **instância independente por serviço**; sem acesso cruzado (plano §3.2).

Controle	Como igualar
Lógica de domínio	**módulo/biblioteca compartilhada** consumida pelas duas variantes.
Contratos de negócio	mesmas entradas, mesmas regras de rejeição, mesmos estados finais.
Topologia/rede	mesmos limites e localização; **uma instância por serviço** no baseline.
Carga, dados e seed	mesmos geradores e mesma **seed**.
Aquecimento	mesmo procedimento antes das medições.

Critérios objetivos de equivalência funcional [R] (plano §3.8): mesmos casos de negócio; mesmos estados finais (sucesso, rejeição, compensação); mesmos **invariantes** de consistência (**confirmados** — ver §3.4); mesma carga; mesma matriz de falhas; **mesma suíte de testes funcionais** aprovada nas duas versões.

> **[R]** A equivalência deve ser **demonstrada** no relatório (testes funcionais iguais nas duas variantes), não apenas afirmada.

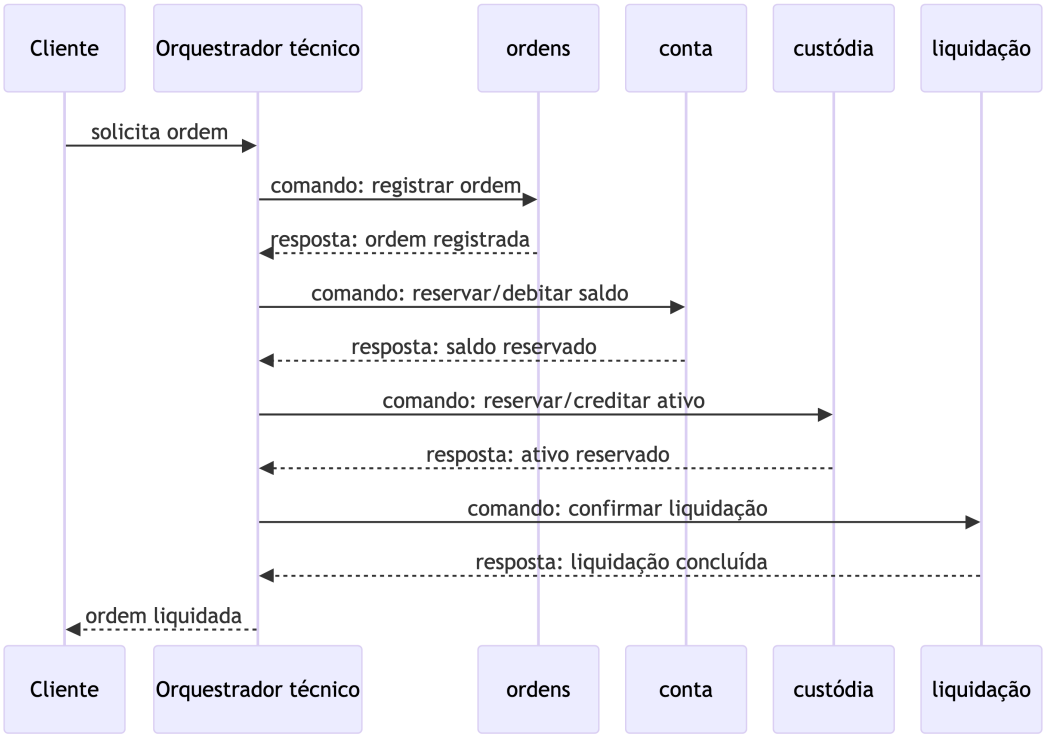
---

2. Cenário de liquidação de ordem

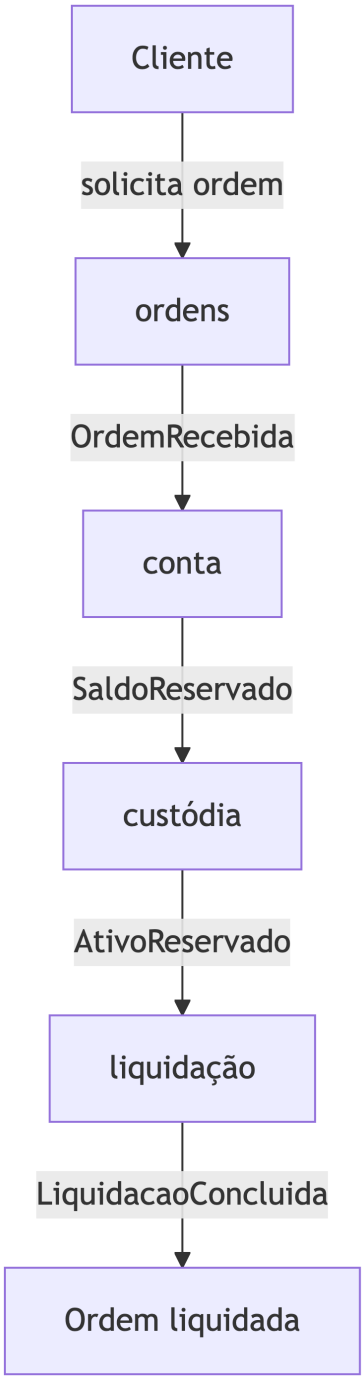
[R] Quatro **serviços de domínio** — `ordens`, `conta`, `custódia`, `liquidação` — e **um orquestrador técnico** presente **apenas** na variante orquestrada (plano §3.1; relatório de validação §7.1).

Componente	Papel no fluxo	Estado de domínio	Observação
`ordens`	recebe a ordem e a registra (entrada do fluxo)	ordem e seu ciclo de vida	primeiro passo; falha aqui → **rejeição sem compensação a jusante**
`conta`	**reserva/debita saldo**	saldo/reservas	compensável
`custódia`	**reserva/credita ativo**	posição/reservas de ativo	compensável
`liquidação`	**confirma a liquidação** e encerra	registro de liquidação	passo final
orquestrador técnico	conduz passos e compensações (**só na orquestrada**)	**apenas** estado de coordenação/Saga — **sem** regra de negócio	não participa do domínio

[**R**] O **orquestrador** não possui regra de negócio nem estado de domínio; os **quatro serviços de domínio são os mesmos** nas duas variantes (plano §3.1). Os **nomes** acima são os já usados no plano; qualquer nome de contrato, tabela ou campo adicional é `[A CONFIRMAR]`.



Saga coreografada — fluxo conceitual por eventos/fatos (caminho feliz):



[**R**] Diagramas **conceituais** (adaptados do plano §3.1.2). **Não** fixam endpoints, *paths* ou protocolos; servem para alinhar o desenho, não para especificar a implementação.

3. Caso feliz: passo a passo, estados, identificadores e invariantes

Os **estados** (§3.3) e os **invariantes** (§3.4) foram **confirmados** nesta revisão ([**C**]). Os **identificadores** (§3.2) e os demais parâmetros de implementação (nível de isolamento, `lock_timeout`, regra de rejeição, idempotência) permanecem `[A CONFIRMAR]` até o protocolo aprovado.

3.1 Passo a passo do caminho feliz

#	Passo	Serviço	Efeito de domínio	Estado final candidato
1	Receber ordem	`ordens`	ordem registrada	`ORDEM_RECEBIDA [A CONFIRMAR]`
2	Reservar/debitar saldo	`conta`	saldo reservado/debitado	`SALDO_RESERVADO [A CONFIRMAR]`
3	Reservar/creditar ativo	`custódia`	ativo reservado/creditado	`ATIVO_RESERVADO [A CONFIRMAR]`
4	Confirmar liquidação	`liquidação`	liquidação registrada	`LIQUIDACAO_CONCLUIDA [A CONFIRMAR]`

Na variante orquestrada: o orquestrador **comanda** cada passo, **aguarda a resposta** e decide o próximo passo ou a compensação (plano §3.1.1).

Na variante coreografada: cada serviço **publica um fato** e o próximo **reage**; a coordenação é emergente (plano §3.1.1).

3.2 Identificadores de correlação e idempotência (candidatos)

[**R**] Proposta inicial — **nomes e propagação `[A CONFIRMAR]`**:

Identificador	Função pretendida	Observação
`sagaId [A CONFIRMAR]`	identificar a execução da Saga	equivalente ao `orderId` no fluxo? `[A CONFIRMAR]`
`orderId [A CONFIRMAR]`	chave de negócio da ordem	usado também para idempotência de negócio
`correlationId [A CONFIRMAR]`	correlacionar mensagens do mesmo fluxo	propagação via broker `[A CONFIRMAR]`

Identificador	Função pretendida	Observação
`causationId [A CONFIRMAR]`	ligar cada mensagem à sua causa	apoio a diagnóstico (RQ3, condicional)
`messageId [A CONFIRMAR]`	identidade da mensagem	base para deduplicação
`idempotencyKey [A CONFIRMAR]`	chave de deduplicação aplicacional	consumidor deduplica `messageId` + chave de negócio

[**R**] A **idempotência** de retries, reenvios e **compensações** é requisito (plano §3.5). O **mecanismo concreto** (tabela de deduplicação, chave única, *outbox*, etc.) é `[A CONFIRMAR]`.

3.3 Estados por serviço (confirmados)

[**C**] Modelo de estados dos quatro serviços:

- **`ordens`:** `RECEBIDA` → (`LIQUIDADA` | `REJEITADA`).
- **`conta`:** `SALDO_RESERVADO` → (`CONFIRMADO` | `RESERVA_DESFEITA`).
- **`custódia`:** `ATIVO_RESERVADO` → (`CONFIRMADO` | `RESERVA_DESFEITA`).
- **`liquidação`:** `CONFIRMADA` (ou ausente, em caso de falha).

A modelagem (máquina de estados, nomes de status e transições) é **idêntica em semântica nas duas variantes**. A **regra/critério de rejeição** permanece `[A CONFIRMAR]` (§3.4, §11.1).

3.4 Invariantes de consistência (confirmados)

[**C**] Invariantes a preservar após a conclusão (inclusive compensações):

- **I1:** nenhuma ordem em `LIQUIDADA` sem as reservas correspondentes de `conta` e `custódia` efetivadas.
- **I2:** não existe reserva de saldo ativa em `conta` sem ordem correspondente não rejeitada.
- **I3:** não existe reserva de ativo ativa em `custódia` sem reserva de saldo compatível em `conta`.
- **I4 (consistência final):** ao final do processamento (incluindo compensações), os estados dos serviços são **mutuamente compatíveis** segundo os invariantes, dentro da **janela de convergência**.

[**C**] Invariantes de concorrência/idempotência:

- **IC1 — saldo disponível:** cada reserva em `conta` só se efetiva se houver **saldo disponível suficiente**; reservas concorrentes da mesma conta **não** podem deixar o saldo disponível negativo (sem sobrevenda de saldo).
- **IC2 — posição:** cada reserva em `custódia` só se efetiva se houver **posição disponível suficiente**; reservas concorrentes do mesmo ativo **não** podem deixar a posição disponível negativa (sem sobrevenda de posição).
- **IC3 — `UNIQUE(orderId):`** a chave de negócio `orderId` é **única** em `ordens`; reenvios/duplicatas da mesma ordem **não** criam registro nem efeito duplicado (base da idempotência de negócio).
- **IC4 — compensação por reserva:** as compensações **desfazem a reserva** previamente criada (liberam saldo/posição), em vez de movimentar valores físicos; após a compensação **não** resta reserva ativa.

  [**A CONFIRMAR**] a **janela de convergência** de consistência (plano §3.6), o **nível de isolamento** do PostgreSQL, o `lock_timeout`, a **regra de rejeição** e o **mecanismo de idempotência** (§3.5, §7, §11.1). Nada aqui é resultado.

3.5 Concorrência inter-Saga e ausência de isolamento

[**C**] **(premissa de desenho)** A **Saga** e a **máquina de estados** associada **não garantem isolamento** entre execuções concorrentes: duas ou mais Sagas sobre a **mesma conta/posição** podem intercalar leituras e escritas, de modo que **efeitos parciais** de uma sejam observados por outra (fonte fundacional: Garcia-Molina; Salem, 1987 — ver §11.2).

Consequências para o protocolo:

- A ausência de isolamento é **inerente ao padrão** e **não** é eliminada por nenhuma das variantes (coreografia ou orquestração).
- [**R**] Para honrar **IC1/IC2** sob concorrência, recomenda-se o **controle de concorrência no próprio banco de cada serviço**: **transação local** por passo
  - `UPDATE` **condicional** (ex.: `... WHERE saldo_disponivel >= :valor`) + **restrição** `UNIQUE(orderId)` — **de forma idêntica nas duas variantes** e **sem lock mantido no orquestrador** (o orquestrador mantém apenas estado de coordenação, sem regra de negócio nem lock de domínio).
- [**A CONFIRMAR**] o **nível de isolamento** exato, o `lock_timeout`, a **regra de rejeição** e o **mecanismo de idempotência** (retries/duplicatas).

---

4. Protocolo sugerido de coordenação

[**R**] Proposta inicial. Nomes de comandos/respostas/eventos e seus contratos são `[A CONFIRMAR]`. **Preserva-se a mesma regra de domínio** nas duas variantes; muda **apenas** o mecanismo de coordenação. O **RabbitMQ** é o **broker comum** (plano §3.2; `docs/confirmacoes.md`, registro interno).

4.1 Saga orquestrada — comandos e respostas (candidatos)

[**R**] Comunicação **imperativa e ponto a ponto**; o orquestrador mantém o estado da Saga e decide as compensações (plano §3.1.1).

Comando (candidato)	Destino	Resposta de sucesso (candidata)	Resposta de falha (candidata)
`RegistrarOrdem [A CONFIRMAR]`	`ordens`	`OrdemRegistrada [A CONFIRMAR]`	`FalhaRegistrarOrdem [A CONFIRMAR]`
`ReservarSaldo [A CONFIRMAR]`	`conta`	`SaldoReservado [A CONFIRMAR]`	`SaldoRecusado [A CONFIRMAR]`
`ReservarAtivo [A CONFIRMAR]`	`custódia`	`AtivoReservado [A CONFIRMAR]`	`AtivoRecusado [A CONFIRMAR]`
`ConfirmarLiquidacao [A CONFIRMAR]`	`liquidação`	`LiquidacaoConcluida [A CONFIRMAR]`	`LiquidacaoFalhou [A CONFIRMAR]`
`DesfazerReservaSaldoconta [A CONFIRMAR]`		`ReservaSaldoDesfeita [A CONFIRMAR]`	`[A CONFIRMAR]`
`DesfazerReservaAtivocustódia [A CONFIRMAR]`		`ReservaAtivoDesfeita [A CONFIRMAR]`	`[A CONFIRMAR]`

[**A CONFIRMAR**] transporte (fila/exchange), timeouts de resposta, e como o orquestrador representa o estado da Saga. **Não** fixar aqui.

4.2 Saga coreografada — eventos/fatos (candidatos)

[**R**] Cada serviço publica **fatos de domínio**; os demais reagem, **sem** coordenador central (plano §3.1.1).

Evento/fato (candidato)	Publicado por	Reação esperada (candidata)
`OrdemRecebida [A CONFIRMAR]`	`ordens`	`conta` reserva/debita saldo
`SaldoReservado [A CONFIRMAR]`	`conta`	`custódia` reserva/credita ativo

Evento/fato (candidato)	Publicado por	Reação esperada (candidata)
`AtivoReservado [A CONFIRMAR]`	`custódia`	`liquidação` confirma
`LiquidacaoConcluida [A CONFIRMAR]`	`liquidação`	encerra o fluxo
`SaldoRecusado` / `AtivoRecusado [A CONFIRMAR]`	`conta` / `custódia`	dispara compensação a montante
`LiquidacaoFalhou [A CONFIRMAR]`	`liquidação`	dispara compensação em cascata
`ReservaSaldoDesfeita` / `ReservaAtivoDesfeita [A CONFIRMAR]`	`conta` / `custódia`	avança a compensação
`OrdemRejeitada [A CONFIRMAR]`	`ordens`	estado terminal de rejeição

[**A CONFIRMAR**] nomes finais, payloads, tópicos/filas no RabbitMQ e como os consumidores garantem idempotência. **Não** fixar aqui.

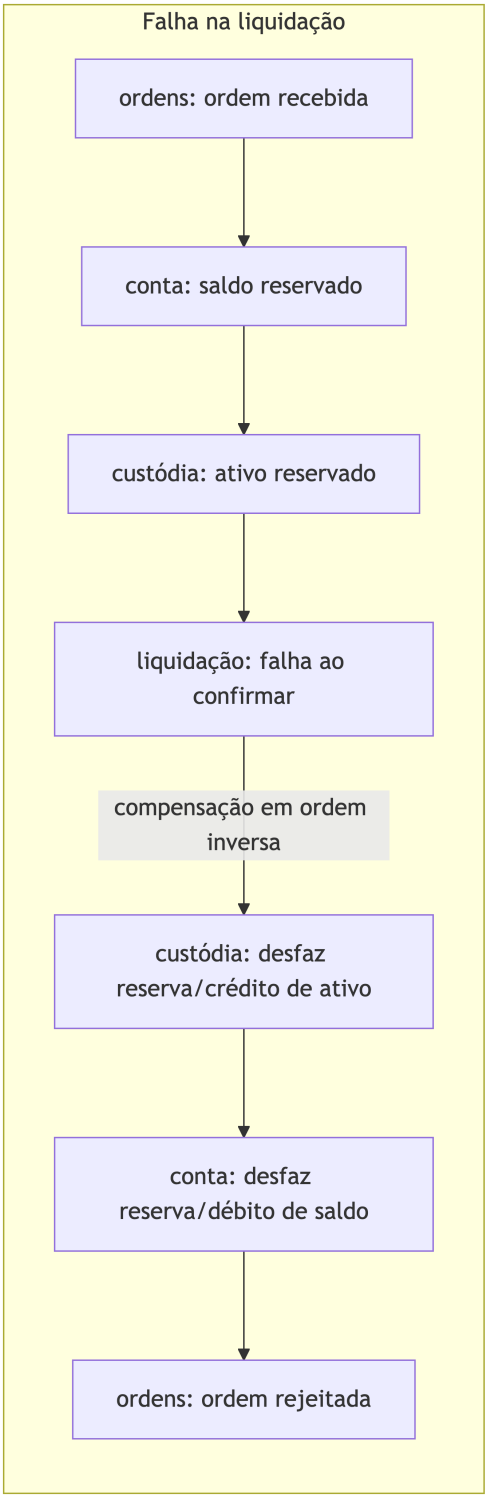
4.3 Brokers e semântica (comum às duas variantes)

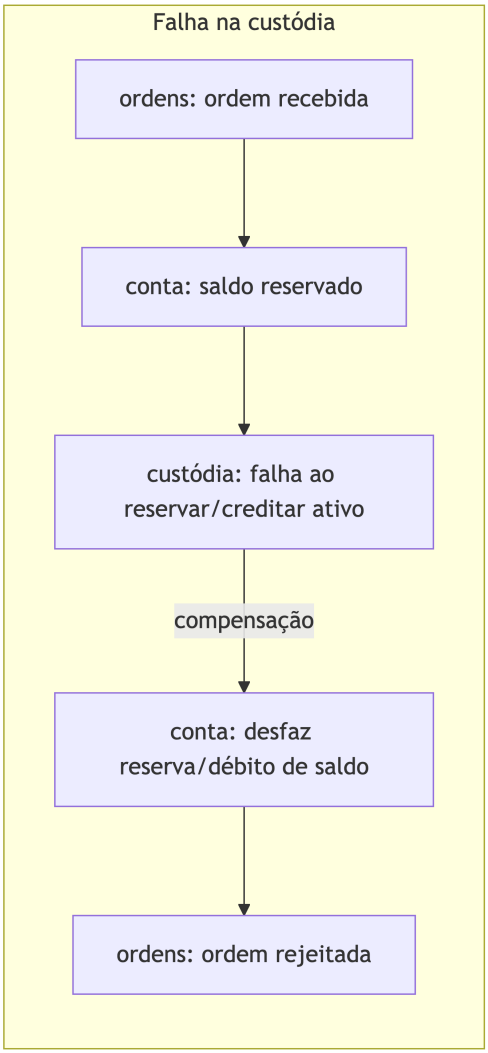
- [**R**] **Uma única** infraestrutura de broker (**RabbitMQ**) para as duas abordagens (plano §3.2).
- [**R**] O protocolo deve **documentar** a **semântica de entrega** adotada (*at-least-once* como hipótese de trabalho `[A CONFIRMAR]`), o uso de **publisher confirms** e de **acknowledgements** de consumidor, e a configuração de **filas/exchanges** — **sem fixar** escolhas ainda não confirmadas.
- [**A CONFIRMAR**] a configuração exata (durabilidade, *prefetch*, retry/DLX, filas de compensação). Ver §9.1 para o papel do RabbitMQ.

---

5. Compensações em ordem inversa e resultado esperado

[**R**] Desfazimento **em ordem inversa** (plano §3.1.1): falha em `custódia` desfaz `conta`; falha em `liquidação` desfaz `custódia` e depois `conta`.


Falha na liquidação
ordens: ordem recebida
conta: saldo reservado
custódia: ativo reservado
liquidação: falha ao confirmar
compensação em ordem inversa
custódia: desfaz reserva/crédito de ativo
conta: desfaz reserva/débito de saldo
ordens: ordem rejeitada


Falha na custódia
ordens: ordem recebida
conta: saldo reservado
custódia: falha ao reservar/creditar ativo
compensação
conta: desfaz reserva/débito de saldo
ordens: ordem rejeitada

5.1 Tabela de resultado esperado por ponto de falha

Ponto de falha	Passos já efetivados	Compensações em ordem inversa	Estado final esperado
`ordens` (passo 1)	nenhum	nenhuma (nada a jusante)	**Rejeição sem compensação**
`conta` (passo 2)	`ordens`	desfaz efeito parcial em `conta`	**Rejeição**; sem saldo reservado
`custódia` (passo 3)	`ordens`, `conta`	`conta` (desfaz reserva de saldo)	**Rejeitada**; sem saldo/ativo reservado
`liquidação` (passo 4)	`ordens`, `conta`, `custódia`	`custódia` → `conta`	**Rejeitada**; sem saldo/ativo reservado

[**A CONFIRMAR**] o critério de sucesso de cada compensação é a **restauração do estado anterior** compatível com os invariantes (§3.4) dentro da janela de convergência; a verificação exata é `[A CONFIRMAR]`.

---

6. Matriz de casos de erro/falha (proposta)

[**R**] Seis tipos de falha já definidos no plano §3.5 — rotulados aqui **C1–C6** —, aplicados de forma **igual** às duas abordagens. **Todos os parâmetros permanecem `[A CONFIRMAR]`** — nenhuma ferramenta de injeção é fixada aqui (plano §3.5). Os **pontos de injeção são candidatos**, não decisões.

6.1 Casos de falha

#	Caso	Ponto de injeção candidato	Efeito esperado	Compensação/reprocessamento	Critério de sucesso	Parâmetros `[A CONFIRMAR]`
C1	**Indisponibilidade de serviço** (queda de participante)	derrubar instância de um serviço (`conta`, `custódia` ou `liquidação`) durante o passo	passo não responde; fluxo interrompido	retries limitados (§7) e, esgotados, compensação a montante	estado final consistente (invariantes §3.4); sem efeitos órfãos	duração da queda, serviço-alvo, *timeout*
C2	**Latência/timeout**	atraso injetado em um passo antes/depois do efeito local	passo ultrapassa *timeout*; possível retry/compensação	retry com backoff; se o efeito já ocorreu, idempotência evita duplicação	consistência final; sem duplicar efeitos	atraso, *timeout*, ocorrência em relação ao efeito
C3	**Rejeição de negócio**	negativa semântica em `conta`/`custódia` (ex.: saldo/ativo insuficiente)	falha semântica sem falha de infraestrutura	compensação a montante em ordem inversa	rejeição com estado anterior restaurado	regra de negócio exata, passo-alvo
C4	**Duplicação de mensagem**	publicar/entregar a mesma mensagem mais de uma vez	consumidor recebe duplicata	deduplicação idempotente no consumidor	nenhum efeito duplicado (invariantes preservados)	taxa de duplicação, ponto
C5	**Perda/indisponibilidade de mensagem**	descartar/indisponibilizar mensagem entre publicação e consumo	passo seguinte não é disparado	reenvio/recuperação (retry, DLQ/outbox — §7)	fluxo retoma ou compensa; consistência final	ponto de perda, janela, mecanismo de recuperação

#	Caso	Ponto de injeção candidato	Efeito esperado	Compensação/reprocessamento	Critério de sucesso	Parâmetros `[A CONFIRMAR]`
C6	**Falha parcial no meio da Saga**	falha após 1–2 passos efetivados (ex.: em `custódia`)	efeitos parciais já persistidos	compensações **em cascata** (ordem inversa)	todos os efeitos parciais desfeitos; invariantes válidos	ponto, instante, duração

6.2 Subcaso candidato — crash após commit local/publicação

[R] **Subcaso candidato** (não implementado): falha do processo **depois** do *commit* local do serviço e **antes/ao** publicar a mensagem (ou vice-versa), cenário clássico de escrita dupla. **Não** é tratado como implementado nem como decidido.

- **Efeito esperado:** efeito local persistido sem mensagem publicada (ou mensagem publicada sem o efeito estar confirmado), dependendo do instante.
- **Compensação/reprocessamento:** depende de mecanismo de publicação confiável e **idempotência**; `[A CONFIRMAR]` se o projeto adotará *outbox* transacional, *transactional messaging* ou equivalente (§7).
- **Critério de sucesso:** nenhum efeito órfão persistido ao final; invariantes válidos.
- **Parâmetros `[A CONFIRMAR]`:** instante do crash (antes/depois do *commit* local e da publicação), serviço-alvo, mecanismo de recuperação.

[R] A **injeção de falhas** será definida **após o amadurecimento do caminho feliz** (plano §3.5): primeiro as duas variantes operam sem falhas, com contratos e compensações estáveis; só então a matriz é fechada.

---

7. Retries, idempotência, DLQ/outbox (decisões a validar)

[R] **Decisão incorporada (plano §3.5; `docs/confirmacoes.md`, registro interno):** adotar **retries limitados** com **número máximo `N` fixo** (`N` = `[A CONFIRMAR]`) e **backoff exponencial**. Retries, reenvios e compensações devem ser **idempotentes**.

Item	Proposta	Status
Nº máximo de tentativas `N`	fixo, a definir no protocolo	`[A CONFIRMAR]`
Backoff	exponencial	[**R**] (decisão incorporada)
Idempotência	tratamento explícito de mensagens duplicadas	[**R**] requisito; mecanismo `[A CONFIRMAR]`
DLQ	eventual fila de mensagens mortas após esgotar retries	`[A CONFIRMAR]`
Outbox / publicação confiável	eventual *outbox* transacional para o subcaso de crash (§6.2)	`[A CONFIRMAR]`
Política de compensação que falha	tratar compensação não executável (saga parcialmente desfeita)	`[A CONFIRMAR]`

[**E**] O caráter não atômico da Saga e a possibilidade de a **compensação falhar** estão documentados na fonte fundacional (Garcia-Molina; Salem, 1987) — ver `research/fichamentos/fichamento-fundamentos-saga.md`. [**R**] O protocolo deve **declarar explicitamente** o comportamento quando uma compensação não é executável.

Não inventar configuração: nenhum valor de `N`, nenhum limiar de backoff, nenhum nome de fila e nenhuma política de DLQ/outbox é fixado aqui.

---

8. Métricas operacionais — como medir (sem resultados)

[**R**] Definições operacionais a fixar **antes** da coleta (plano §3.6). **Nenhum resultado ou valor experimental** é apresentado. Limiares e janelas são `[A CONFIRMAR]`.

Métrica	Definição operacional (proposta)	Como medir	Instrumento	Limiar/janela
Sucesso	ordem/Saga concluída com todos os passos efetivados e estado final esperado, **sem** compensação	contagem por desfecho da ordem	métricas de aplicação + agregação pós-execução	—
Rejeição	ordem encerrada sem efeitos de domínio remanescentes	contagem por desfecho	idem	—
Compensação correta	proporção de Sagas compensadas cujos serviços voltaram ao estado anterior e satisfazem os invariantes	contagem de compensações verificadas ÷ total de compensações	métricas de aplicação + verificação de invariantes	definição de "correta" `[A CONFIRMAR]`
Consistência final	estados dos serviços mutuamente compatíveis após conclusão (incl. compensações)	verificação de invariantes (§3.4) ao final e na janela	consulta de verificação + traces	**janela de convergência** `[A CONFIRMAR]`
Tempo até consistência	intervalo entre falha/início da compensação e observação do estado consistente	timestamps correlacionados	traces/métricas	janela `[A CONFIRMAR]`
Tempo de recuperação	intervalo entre injeção da falha e retorno ao estado operacional esperado	timestamps	traces/sonda de saúde	início/fim e limiar `[A CONFIRMAR]`

Métrica	Definição operacional (proposta)	Como medir	Instrumento	Limiar/janela
p50/p95/p99	percentis da distribuição de latência **por ordem**	**histogramas** Prometheus + `histogram_quantile`	Prometheus (via métricas dos serviços)	buckets e janelas definidos antes da coleta `[A CONFIRMAR]`
Throughput	ordens/Sagas concluídas por unidade de tempo (RPS concluídas)	taxa de conclusão na janela estável	agregação pós-execução	denominador `[A CONFIRMAR]`
Taxa de erro	proporção de ordens/Sagas que terminam em erro **não compensado**	contagem ÷ total iniciado	métricas de aplicação	denominador exato `[A CONFIRMAR]`
Taxa de compensação	proporção de Sagas compensadas ÷ total iniciado	contagem ÷ total iniciado	métricas de aplicação	denominador exato `[A CONFIRMAR]`
CPU/memória	consumo dos serviços e do orquestrador na janela estável	*scraping* Prometheus	cAdvisor/node exporter `[A CONFIRMAR]`	agregação/unidade `[A CONFIRMAR]`
Mensagens	publicadas × consumidas; nº de duplicatas/perdas/reatribuições	contadores no broker/aplicação	métricas do broker + aplicação	baseline/tolerância `[A CONFIRMAR]`

[**R**] **p95/p99** devem vir de **histogramas** (e não de médias), com **buckets e janelas definidos antes da coleta** (§9.2). Percentis são da **distribuição por ordem** e agregados por execução/campanha (plano §3.6).

[**A CONFIRMAR**] limiares de relevância prática, denominadores e janela de convergência (plano §3.6, §3.9). Nada aqui antecipa resultado.

9. Papel das ferramentas (confirmadas e candidatas)

[**E**] Stack decidida (plano §3.2; `docs/confirmacoes.md`, registro interno): Java/Spring Boot, PostgreSQL, **RabbitMQ**, Docker Compose, **k6** (candidato), Prometheus/OpenTelemetry. [**R**] As ferramentas de mercado abaixo entram **apenas como contexto/ideias** — **nunca** como evidência comparativa (relatório de validação §9; `docs/confirmacoes.md`, registro interno).

9.1 RabbitMQ — broker e comunicação assíncrona

- [**R**] Broker **comum** às duas variantes; comunicação **assíncrona**.
- [**R**] O protocolo deve **documentar** a **semântica de entrega** adotada (hipótese *at-least-once* `[A CONFIRMAR]`), o uso de **publisher confirms** e **acknowledgements** de consumidor, e a **configuração de filas/exchanges** — **sem fixar** escolhas não confirmadas.
- [**A CONFIRMAR**] durabilidade, *prefetch*/QoS, DLX/DLQ, filas de retry e de compensação.
- **Fonte de documentação oficial (verificada):** *RabbitMQ Documentation* — https://www.rabbitmq.com/docs (acesso em 29 set. 2026).

9.2 Prometheus — coleta de métricas

- [**R**] Coleta de métricas de runtime; recomenda-se **histogramas** para p95/p99 e a **definição prévia de buckets e janelas**.
- [**A CONFIRMAR**] buckets, intervalos de *scraping* e janelas de agregação.
- **Fonte de documentação oficial (verificada):** *Prometheus Overview* — https://prometheus.io/docs/introduction/overview/ (acesso em 29 set. 2026).

9.3 Grafana — visualização

- [**E**] Grafana já existe nos servidores de teste (registro interno).
- [**R**] *Dashboards* e *data sources* devem ser **versionados como código** (provisionamento), para reprodutibilidade.
- [**A CONFIRMAR**] quais painéis e quais fontes.
- **Fonte de documentação oficial (verificada):** *Grafana documentation* — https://grafana.com/docs/grafana/latest/ (acesso em 29 set. 2026).

9.4 k6 — candidato a gerador de carga

- [**E**] k6 é **candidato** (decidido como hipótese de ferramenta; o(a) orientador(a) registrou que avaliaria o k6 — registro interno).
- [**R**] Propor um **piloto técnico**: usar **executor de taxa de chegada fixa** (*constant-arrival-rate*) e **verificar que o gerador não se torna o gargalo**.
- [**A CONFIRMAR**] decisão final de adoção, patches de carga, VUs e *thresholds*.
- **Fonte de documentação oficial (verificada):** *Grafana k6 documentation* — https://grafana.com/docs/k6/latest/ (acesso em 29 set. 2026).

9.5 Ferramentas/frameworks de mercado — somente contexto

[**R**] Apresentar **brevemente** como referência da relevância do tema e como **fonte de ideias de implementação**; **não** usar nenhuma delas como evidência comparativa nem como braço experimental (registro interno; relatório de validação §9).

Ferramenta	Papel aqui
AWS Prescriptive Guidance — Saga patterns	contexto industrial de definição e das variantes
Temporal	contexto de motor de workflow/atividades e idempotência
Axon (AxonIQ)	contexto de framework de mensageria/CQRS e Saga
Eventuate (Eventuate Tram/Sagas)	contexto de framework de Saga do mesmo autor de microservices.io

- **Fontes de documentação oficiais (verificadas em 29 set. 2026):**
  - AWS Prescriptive Guidance — *Saga patterns*: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga-patterns.html (URL já registrada no repositório).
  - Temporal — https://docs.temporal.io/.
  - Axon — https://docs.axoniq.io/ (redireciona para `docs.axoniq.io/home/`).
  - Eventuate — https://eventuate.io/.

[**R**] **microservices.io — orientação interna de uso:** conceitos e diagramas de **Chris Richardson / microservices.io** podem ser usados no trabalho acadêmico com **atribuição** ao autor/fonte, **preferência por paráfrase** e **diagramas adaptados** (ex.: *Fonte: Adaptado de Richardson (microservices.io)*). Trata-se de **orientação interna de redação**, **não** de parecer jurídico nem de licença externa verificada.

10. Matriz inicial de execução e checklist de validação

10.1 Matriz inicial de execução (proposta)

[**R**] Estrutura de campanha; todos os parâmetros numéricos são `[A CONFIRMAR]` e serão calibrados no piloto (plano §3.7, §3.8).

Cenário	Abordagens	Carga	Repetições	Aquecimento	Seed/dados	Status
Caminho feliz (*steady-state*)	coreografada / orquestrada	patamares a calibrar	começar com 5 e reavaliar `[A CONFIRMAR]`	mesmo procedimento	mesma seed	**Não executado**
Falhas C1–C6 (§6.1)	coreografada / orquestrada	patamares a calibrar	começar com 5 e reavaliar `[A CONFIRMAR]`	mesmo procedimento	mesma seed	**Não executado**
Subcaso crash (§6.2)	coreografada / orquestrada	`[A CONFIRMAR]`	`[A CONFIRMAR]`	mesmo procedimento	mesma seed	**Candidato — não implementado**

- [**R**] **Ordem aleatorizada** dos cenários; ambiente isolado; **nenhum experimento coexecutado**; rodadas de aquecimento descartadas (plano §3.8).
- [**R**] Dados brutos imutáveis por execução, com metadados de **seed, versão, ambiente e timestamp** (plano §3.8).

10.2 Checklist de validação do protocolo

☐ [**R**] Fluxo-base (4 passos) e papéis dos serviços validados.
☐ [**R**] Papel do **orquestrador técnico** (só na orquestrada) validado.
☐ [**R**] **Regras de equivalência** (§1.3) aprovadas e demonstradas por suíte funcional única.
☐ [**R**] Nomes/contratos de **comandos, respostas, eventos e fatos** fechados (§4).
☐ [**R**] Identificadores de correlação/idempotência definidos (§3.2).
☒ [**C**] **Estados** dos serviços (§3.3) e **invariantes** I1–I4/IC1–IC4 (§3.4) confirmados.

- ☐ [**R**] **Janela de convergência**, **nível de isolamento**, `lock_timeout`, **regra de rejeição** e **idempotência** definidos (§3.4–§3.5).
- ☐ [**R**] Compensações em ordem inversa e resultado esperado por falha validados (§5).
- ☐ [**R**] Matriz de falhas fechada **após** amadurecer o caminho feliz (§6).
- ☐ [**R**] Política de retries (`N`), backoff, idempotência, DLQ/outbox definida (§7).
- ☐ [**R**] Definições operacionais das métricas e **buckets/janelas** definidos antes da coleta (§8; §9.2).
- ☐ [**R**] Papéis das ferramentas confirmados; k6 validado quanto a não ser gargalo (§9).
- ☐ [**R**] Plano de análise (IC, tamanho de efeito, regra de decisão) e nº de repetições acordados (plano §3.9).

---

11. Itens ainda a confirmar e referências

11.1 Itens `[A CONFIRMAR]`

- **Nomes/contratos** de comandos, respostas, eventos, fatos e identificadores (§3.2, §4–§5); os **estados** (§3.3) e os **invariantes** I1–I4/IC1–IC4 (§3.4) são **[C]** (confirmados).
- **Janela de convergência**, **nível de isolamento**, `lock_timeout`, **regra de rejeição** e **mecanismo de idempotência** (§3.4–§3.5).
- **Configuração do broker** (semântica, confirms, filas, DLX/DLQ) (§4.3, §9.1).
- **Mecanismo de injeção de falhas** e parâmetros de cada caso (§6).
- `N` **de retries**, backoff, idempotência, **DLQ/outbox** (§7).
- **Limiares, denominadores e janelas** das métricas; **buckets** Prometheus (§8).
- **Adoção final do k6** e patches de carga; **painéis Grafana** (§9.3–9.4).
- **Versões exatas** de linguagem, framework, broker, banco e bibliotecas e **nome da pasta do experimento** (plano §5.1).
- **Política de versionamento** do repositório e forma/licença de publicação do artefato (plano §5.1, §8) — decisões internas de protocolo.

Nada nesta seção é resultado. Nenhum experimento foi executado.

11.2 Referências acadêmicas e documentais (ABNT)

Fundacional e de transações

- GARCIA-MOLINA, Hector; SALEM, Kenneth. **Sagas**. In: ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA, 1987, New York. *Proceedings […].* New York: ACM, 1987. p. 249–259. DOI: 10.1145/38713.38742.

- GRAY, Jim. **Notes on data base operating systems**. In: BAYER, R.; GRAHAM, R. M.; SEGMULLER, G. (ed.). *Operating systems: an advanced course.* Berlin: Springer, 1978. p. 393–481. (Lecture Notes in Computer Science, v. 60). DOI: 10.1007/3-540-08755-9_9.
- SKEEN, Dale. **Nonblocking commit protocols**. In: ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA, 1981. p. 133–142. DOI: 10.1145/582318.582339.
- GRAY, Jim; LAMPORT, Leslie. **Consensus on transaction commit**. *ACM Transactions on Database Systems*, v. 31, n. 1, p. 133–160, 2006. DOI: 10.1145/1132863.1132867.

Fundamentação conceitual/industrial

- RICHARDSON, Chris. **Microservices patterns: with examples in Java**. Shelter Island: Manning, 2018. cap. 4, p. 110–145. ISBN 9781617294549.
- RICHARDSON, Chris. **Pattern: Saga**. *microservices.io*, 2024. Disponível em: https://microservices.io/patterns/data/saga.html. Acesso em: 29 set. 2026. *(Orientação interna de uso: atribuição, paráfrase e diagramas adaptados.)*
- AMAZON WEB SERVICES. **Saga patterns**. *AWS Prescriptive Guidance*. Disponível em: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga-patterns.html. Acesso em: 29 set. 2026.
- MICROSOFT. **Saga distributed transactions pattern**. *Azure Architecture Center*. Disponível em: https://learn.microsoft.com/en-us/azure/architecture/patterns/saga. Acesso em: 29 set. 2026.

Estudos comparativos (linha de base)

- RUDRABHATLA, C. K. **Comparison of event choreography and orchestration techniques in microservice architecture**. *International Journal of Advanced Computer Science and Applications*, v. 9, n. 8, p. 18–22, 2018. DOI: 10.14569/IJACSA.2018.090804.
- KRISTIANTO, Handi; ZAHRA, Amalia. **Performance analysis of choreography and orchestration in microservices architecture**. *Journal of Theoretical and Applied Information Technology*, v. 99, n. 18, p. 4220–4230, 2021.
- AYDIN, Sahin; ÇEBI, Cem Berke. **Comparison of choreography vs orchestration based saga patterns in microservices**. In: INTERNATIONAL CONFERENCE ON ELECTRICAL, COMPUTER AND ENERGY TECHNOLOGIES (ICECET), 2022, Prague. *Proceedings […].* 2022. p. 1–6. DOI: 10.1109/ICECET55527.2022.9872665.
- MARQUES, David Miguel Sousa. **Microservices orchestration vs. choreography: a comparison and analysis**. 2024. Dissertação (Mestrado em Engenharia Informática) — Instituto Superior de Engenharia do Porto, Instituto Politécnico do Porto, Porto, 2024. Handle: 10400.22/27107.
- NADEEM, Anas; MALIK, Muhammad Zubair. **A case for microservices orchestration using workflow engines**. In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING — NEW IDEAS AND EMERGING RESULTS (ICSE-NIER), 2022. p. 6–10. DOI: 10.1145/3510455.3512777.

Metodologia

- WOHLIN, Claes et al. **Experimentation in software engineering**. 2nd ed. Berlin: Springer, 2012. DOI: 10.1007/978-3-642-29044-2.

Documentação de ferramentas (URLs oficiais verificadas em 29 set. 2026)

- RABBITMQ. **RabbitMQ Documentation**. Disponível em: https://www.rabbitmq.com/docs. Acesso em: 29 set. 2026.
- PROMETHEUS. **Overview**. Disponível em: https://prometheus.io/docs/introduction/overview/. Acesso em: 29 set. 2026.
- GRAFANA LABS. **Grafana documentation**. Disponível em: https://grafana.com/docs/grafana/latest/. Acesso em: 29 set. 2026.
- GRAFANA LABS. **Grafana k6 documentation**. Disponível em: https://grafana.com/docs/k6/latest/. Acesso em: 29 set. 2026.
- TEMPORAL. **Temporal Platform Documentation**. Disponível em: https://docs.temporal.io/. Acesso em: 29 set. 2026.
- AXONIQ. **Axon documentation**. Disponível em: https://docs.axoniq.io/. Acesso em: 29 set. 2026.
- EVENTUATE. **Eventuate**. Disponível em: https://eventuate.io/. Acesso em: 29 set. 2026.

[**R**] As URLs de ferramentas acima foram **verificadas** como páginas oficiais na data de acesso; **não** são evidência comparativa. As versões de ferramentas não foram fixadas (nenhuma versão é afirmada como decisão).

Documento de trabalho — proposta inicial — não oficial. Elaborado a partir do plano de trabalho (§§3.1–3.10 e apêndices), dos fichamentos de fundamentos, 2PC e estudos comparativos e do relatório de validação de escopo. Nenhum experimento foi executado e nenhum resultado é afirmado. Não altera o plano nem outros arquivos existentes. Sujeito a validação do(a) orientador(a) e à conferência da fonte primária antes de uso no TCC (AGENTS.md, seções 5, 7 e 8).